

Reviewer Pack — Minimal Local Index and Communication Complexity Rank Variance...

Entry 9b1bf713078f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 07:56:32 UTC

Minimal Local Index and Communication Complexity Rank Variance

Entry ID: 9b1bf713078f **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 07:56:32 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Local Systems) **Field B** (complexity object): Communication Complexity (Matrix Rank Variance)

Statement:

The minimal local index of a communication complexity matrix, defined as the minimum order of its characteristic polynomial over the complex numbers, is linearly correlated with the variance of its rank, such that $\log(\min_local_index) = \Theta(\text{Var}(\text{rank}))$ for all matrices with $n \leq 40$ variables.

Rationale (proposer's reasoning):

Algebraic topology provides tools to study the intrinsic structure of spaces, which might reveal hidden properties of communication complexity matrices. Local systems in algebraic topology could potentially expose patterns in matrix rankings that are not evident through classical linear algebra approaches. This correlation could shed light on the intrinsic difficulty of solving certain communication complexity problems.

Taxonomy category: Local Systems in Communication Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9551ce3f2e6a027b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between $\log(\min_local_index)$ and $\text{Var}(\text{rank})$ across all matrices with $n \leq 40$ variables is greater than or equal to 0.7.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local index" AND "communication complexity" AND matrix rank variance" - "Algebraic Topology" AND characteristic polynomial" AND communication complexity" - "Communication Complexity" AND matrix rank variance" AND algebraic topology"

Top relevant hits considered: - [http://arxiv.org/abs/0808.1395v4] Algebraic topology from geometric viewpoint - [http://arxiv.org/abs/1601.00254v2] Euler characteristic reciprocity for chromatic, flow and order polynomials - [http://arxiv.org/abs/math/9912085v1] TQFT - a new direction in algebraic topology - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/1912.12844v1] Variance Reduced Local SGD with Lower Communication Complexity - [http://arxiv.org/abs/2410.20094v1] The Communication Complexity of Approximating Matrix Rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below the pivot
        for j in range(i+1, n):
            factor = Fraction(matrix[j][i], matrix[i][i])
            for k in range(n):
                matrix[j][k] -= factor * matrix[i][k]

def characteristic_polynomial(matrix):
    n = len(matrix)
    coeff = [Fraction(1)]
    for i in range(n):
        new_coeff = [0] * (n - i + 1)
        for j in range(i+1, n):
```

```

        for k in range(j):
            if k < i:
                new_coeff[j-k] += matrix[i][k] * coeff[-j+k]
            else:
                new_coeff[j-k] -= matrix[i][k] * coeff[-j+k-1]
        coeff = new_coeff
    return coeff

def min_local_index(coeff):
    n = len(coeff) - 1
    if n == 0: return 0
    for i in range(1, n+1):
        if all(abs(coeff[j]) < abs(coeff[j+i]) for j in range(n-i+1)):
            return i
    return n

def variance(lst):
    mean = sum(lst) / len(lst)
    return sum((x - mean) ** 2 for x in lst) / len(lst)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in range(5, 41):
        rank_variances = [random.randint(1, 100) for _ in range(30)]
        for rank_variance in rank_variances:
            matrix = [[random.randint(-10, 10) for _ in range(n)] for _ in range(n)]
            gaussian_elimination(matrix)
            coeff = characteristic_polynomial(matrix)
            min_local = min_local_index(coeff)
            results.append((n, min_local, rank_variance))

    if not results:
        return {
            "metric_name": "log_min_local_index",
            "metric_value": 0,
            "instances_tested": 0,
            "n_max": 40,
            "conjecture_holds": False,
            "counterexample": "no_data"
        }

    log_msl = [math.log(result[1]) for result in results]
    var_rank = [result[2] for result in results]
    correlation_coefficient = sum((log_msl[i] - mean(log_msl)) * (var_rank[i] - mean(var_rank)) for i in range(len(log_msl))) / len(log_msl)

    return {
        "metric_name": "log_min_local_index",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": 40,
        "conjecture_holds": abs(correlation_coefficient) >= 0.7,
        "counterexample": ""
    }

def mean(lst):

```

```

    return sum(lst) / len(lst)

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{trial_result['metric_name']}\", \"metric_value\": {trial_result['metric_value']}}}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        RESULT = "SUPPORTED"
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        RESULT = f"FALSIFIED counterexample=\"first_failing_seed\" first_failing_seed={first_failing_seed}"
    else:
        RESULT = "INCONCLUSIVE"

    print(f"RESULT: {RESULT} mean={mean([result['metric_value'] for result in results])} std=0 sup={max([result['metric_value'] for result in results])}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_alb71144.py", line 104, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_alb71144.py", line 68, in run_trial
    coeff = characteristic_polynomial(matrix)
            ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_alb71144.py", line 44, in characteristic_polynomial
    new_coeff[j-k] -= matrix[i][k] * coeff[-j+k-1]
                        ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the Pearson correlation coefficient could not be calculated. | next: Re-run the test to ensure it completes successfully and produces the necessary data for calculating the Pearson correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15667
2	propose	ollama_remote	glm4:latest	0	0	13393
3	propose	ollama_remote	glm4:latest	0	0	13767
4	preregistration	ollama_remote	glm4:latest	0	0	12880
5	novelty	ollama_remote	glm4:latest	0	0	8135
6	novelty	ollama_remote	glm4:latest	0	0	9855
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19486
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11761
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15055
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13577
11	judge	ollama_remote	glm4:latest	0	0	19515

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 153092 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/9b1bf713078f.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/9b1bf713078f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9b1bf713078f.tar.gz (if generated)